Week 10
SAC, TD3, and CrossQ for Continuous Control

STUDY-RL — a self-paced Deep RL curriculum

October 6, 2026

Contents

Learning objectives 3

1 Motivation and intuition 4

2 Mathematical formulation 5
- 2.1 Stochastic vs. deterministic policies . . . 5
- 2.2 Maximum entropy RL . . . 5
- 2.3 Tanh-squashed Gaussian + Jacobian correction . . . 6
- 2.4 Automatic temperature tuning . . . 6
- 2.5 TD3 . . . 6
- 2.6 CrossQ (2024) — target nets considered harmful . . . 7

3 Algorithms 8

4 Implementation walkthrough 9
- 4.1 Tanh-squashed Gaussian (the right way) . . . 9
- 4.2 SAC critic update . . . 9

5 Empirical results 10

6 Safety angle 11

7 Matching application 12

8 Exercises 13

9 Further reading 14

Learning objectives

Study companion. Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

Key Idea

By the end of this week you will be able to:

1. Contrast stochastic and deterministic policy families and identify when each yields a lower-variance gradient.
2. Derive the SAC (soft actor-critic) objective from the maximum-entropy MDP formulation; state the soft Bellman equations and the closed-form soft policy.
3. Implement the tanh-squashed Gaussian policy, including the Jacobian correction to its log-density, and explain why the correction is essential.
4. Train SAC on Pendulum-v1 (or MuJoCo Hopper) with automatic temperature tuning; plot return and entropy.
5. Specify TD3 (twin critics, target policy smoothing, delayed updates) and explain its role as the deterministic-policy counterpart to SAC.
6. Summarise CrossQ (2024) and the SimBa block (2025) as the off-policy continuous-control SOTA, including why CrossQ drops target networks.

1 Motivation and intuition

PPO is on-policy: it reuses each rollout for several optimisation epochs, then normally replaces it with fresh data. For continuous control with expensive simulators (or robots), this is a luxury. Off-policy methods — SAC, TD3, and the modern ensemble variants — amortise every transition across many gradient updates through a replay buffer, and they exploit the reparameterisation trick (week 7) to compute low-variance pathwise gradients for the actor.

Intuition. SAC and TD3 are duals.

- TD3 trains a *deterministic* policy $a = \mu_\theta(s)$. Exploration comes from Gaussian noise added to actions at *rollout* time, not from the policy itself. Stability tricks: twin critics, delayed actor updates, target-action smoothing.
- SAC trains a *stochastic* policy $\pi_\theta(a \mid s) = \tanh(\mathcal{N}(\mu_\theta(s), \sigma_\theta(s)^2))$. Exploration is a property of the policy itself. The critic learns the soft Q-function; the actor maximises a soft-Bellman-corrected entropy-regularised objective.

Both train two critics and use target networks; both update the actor by gradient descent through the critic (pathwise gradient).

Throughput rule of thumb: SAC at UTD=1 sees the same data 8–16 times via the replay buffer; CrossQ at UTD=2 amortises each transition ∼32 times. On a single GPU you can typically get 10–50× more sample-efficient learning out of off-policy than on-policy methods.

Common pitfall

The single most common bug in a SAC implementation is forgetting the tanh-Jacobian correction in the log-probability. The policy will then silently collapse to a near-deterministic edge of the action space, scoring artificially high entropy reported by the un-corrected formula. Verify the density, not its logarithm: $\int_{-1}^{1} \pi(a \mid s)\, da = 1$. For one action dimension and $U \sim \text{Uniform}(-1, 1)$, $2\mathbb{E}[\exp(\log \pi(U \mid s))] = 1$; numerical quadrature is another check.

2 Mathematical formulation

2.1 Stochastic vs. deterministic policies

Stochastic policy gradient (score-function form): $\nabla_\theta J = \mathbb{E}[\nabla_\theta \log \pi_\theta(a \mid s)\, Q^\pi(s,a)]$.

Deterministic policy gradient (Silver 2014):

Theorem 2.1.1 (DPG)**.** *For a deterministic policy $a = \mu_\theta(s)$ and a state distribution d^μ,*

$$\nabla_\theta J(\mu_\theta) = \mathbb{E}_{s\sim d^\mu}\left[\nabla_\theta \mu_\theta(s)\, \nabla_a Q^{\mu_\theta}(s,a)\big|_{a=\mu_\theta(s)}\right].$$

Sketch. Apply the chain rule to $J(\mu_\theta) = \mathbb{E}_{s\sim d^\mu}[Q^{\mu_\theta}(s,\mu_\theta(s))]$. The expectation is over a parameter-dependent d^μ, but the same Bellman-flow trick as in the stochastic PG theorem cancels the implicit $\partial d^\mu/\partial\theta$ contribution; the surviving term is exactly the chain rule applied to the inner $Q(s,\mu_\theta(s))$. □

The deterministic gradient has lower variance (it averages out the exploration noise of the stochastic version) but requires the action space to be continuous and the critic to be differentiable in a.

2.2 Maximum entropy RL

The classical RL objective $J(\theta) = \mathbb{E}[\sum_t \gamma^t r_t]$ is augmented to

$$J_{\text{MaxEnt}}(\theta) = \mathbb{E}\left[\sum_t \gamma^t \big(r_t + \alpha \mathcal{H}(\pi_\theta(\cdot \mid s_t))\big)\right],$$

with $\alpha \geq 0$ the temperature. The optimal policy is no longer deterministic; it is the Gibbs distribution over actions weighted by Q.

Theorem 2.2.1 (Soft Bellman equations, Haarnoja 2018a)**.** *For the entropy-regularised MDP, the soft value function $V^\pi(s) = \mathbb{E}_{a\sim\pi}[Q^\pi(s,a) - \alpha \log \pi(a \mid s)]$, and the soft Bellman backup*

$$\mathcal{T}^\pi Q(s,a) \;=\; r(s,a) + \gamma\, \mathbb{E}_{s'}\big[\mathbb{E}_{a'\sim\pi}[Q(s',a') - \alpha \log \pi(a' \mid s')]\big].$$

The soft-optimal policy is the Boltzmann form $\pi^\star(a \mid s) \propto \exp(Q^\star(s,a)/\alpha)$.

Derivation of the optimal policy. At fixed Q, the actor update solves

$$\pi^{\text{new}}(\cdot \mid s) = \arg\max_\pi \mathbb{E}_{a\sim\pi}[Q(s,a)] + \alpha \mathcal{H}(\pi(\cdot \mid s)).$$

Lagrangian on the simplex $\int \pi(a \mid s)\, da = 1$:

$$\mathcal{L}(\pi,\nu) = \int \pi(a \mid s)\, Q(s,a)\, da - \alpha \int \pi \log \pi\, da - \nu\Big(\int \pi\, da - 1\Big).$$

Functional derivative: $\delta\mathcal{L}/\delta\pi = Q(s,a) - \alpha(\log\pi + 1) - \nu = 0$. Solving, $\pi(a \mid s) = \exp((Q(s,a) - \nu - \alpha)/\alpha)$, and normalising gives the Boltzmann form. The result $V = \alpha \log \int \exp(Q/\alpha)\, da$ is the soft-max (log-sum-exp) over actions. □

2.3 Tanh-squashed Gaussian + Jacobian correction

We need a stochastic policy that:

1. samples in a bounded box $[-1, 1]^{|\mathcal{A}|}$ (continuous-control envs require this);
2. is differentiable through the sample (reparameterisable);
3. has a tractable density.

The standard recipe: $u \sim \mathcal{N}(\mu_\theta(s), \sigma_\theta(s)^2)$, then $a = \tanh(u)$. By the change-of-variables formula,

$$\pi(a \mid s) = \mathcal{N}(u; \mu, \sigma^2) \cdot \left| \frac{du}{da} \right|.$$

Since $u = \tanh^{-1}(a)$, $du/da = 1/(1 - a^2)$, while $da/du = 1 - \tanh^2(u)$. Taking logarithms gives

$$\log \pi(a \mid s) = \log \mathcal{N}(u; \mu, \sigma^2) - \sum_{i=1}^{|\mathcal{A}|} \log(1 - a_i^2).$$

Numerically, $\log(1 - \tanh^2 u) = 2(\log 2 - u - \mathrm{softplus}(-2u))$ is stable.

Common pitfall

Drop the Jacobian and your policy will report $\log \pi$ that is artificially *lower* (because tanh compresses); the entropy bonus $-\alpha \log \pi$ in the actor loss becomes artificially high; the policy will push to the boundary of the action space and saturate. The optimiser will not crash; it will silently converge to a wrong policy.

2.4 Automatic temperature tuning

Haarnoja 2018b makes α a learned parameter by enforcing the constraint

$$\mathbb{E}_{a \sim \pi_\theta}[-\log \pi_\theta(a \mid s)] \geq \mathcal{H}_{\text{target}} = -|\mathcal{A}|.$$

The dual update is

$$\alpha \leftarrow \alpha - \eta_\alpha \nabla_\alpha \alpha \cdot (-\log \pi_\theta(a \mid s) - \mathcal{H}_{\text{target}}),$$

implemented by storing $\log \alpha$ unconstrained and computing $\alpha = \exp(\log \alpha)$. This removes the brittle α hyperparameter and is the single biggest reason SAC is the default off-policy baseline.

2.5 TD3

Twin Delayed DDPG (Fujimoto 2018) fixes three pathologies of DDPG:

1. **Overestimation bias.** Train two critics Q_{ϕ_1}, Q_{ϕ_2}; the target uses $\min(Q_{\phi'_1}, Q_{\phi'_2})$.
2. **Function-approximation error in actor target.** Delay actor updates: update the actor every $d = 2$ critic steps.
3. **Deterministic-policy target sharpness.** Smooth the target action: $a' = \mu_{\theta'}(s') + \mathrm{clip}(\epsilon, -c, c)$, $\epsilon \sim \mathcal{N}(0, \tilde{\sigma}^2)$.

The TD3 target is

$$y = r + \gamma \min_{i=1,2} Q_{\phi'_i}(s', a'), \quad a' = \mu_{\theta'}(s') + \mathrm{clip}(\epsilon, -c, c).$$

2.6 CrossQ (2024) — target nets considered harmful

Bhatt et al. (ICLR 2024) show that with careful BatchNorm placement (BN after every Linear, both critics, train mode during the target computation too), the polyak-averaged target nets can be dropped without instability. Practical gain: $\sim$25% wall-clock improvement and one fewer hyperparameter. The SimBa block (Lee et al. ICLR 2025) builds on this with residual + LayerNorm + RMSNorm and pushes the SOTA further on DMC.

3 Algorithms

SAC with automatic temperature

Hyper-params: replay buffer D of size 10^6, batch $B = 256$, $\gamma = 0.99$, polyak $\tau = 5 \times 10^{-3}$, actor/critic lr $= 3 \times 10^{-4}$, α-lr $= 3 \times 10^{-4}$, $\mathcal{H}_{\text{target}} = -|\mathcal{A}|$.
Init: actor π_θ, critics Q_{ϕ_1}, Q_{ϕ_2} and their polyak copies $Q_{\phi'_1}, Q_{\phi'_2}$.
For each env step:

1. Sample $a \sim \pi_\theta(\cdot \mid s)$, step env, store (s, a, r, s', d).
2. Sample minibatch $\sim D$.
3. Compute target: $\tilde{a} \sim \pi_\theta(\cdot \mid s')$, $\log \tilde{p} = \log \pi_\theta(\tilde{a} \mid s')$,
$$y = r + \gamma(1 - d)\big[\min_i Q_{\phi'_i}(s', \tilde{a}) - \alpha \log \tilde{p}\big].$$
4. Critic update: minimise $\sum_i (Q_{\phi_i}(s, a) - y)^2$.
5. Actor update (reparameterised): minimise $\mathbb{E}[\alpha \log \pi_\theta(a \mid s) - \min_i Q_{\phi_i}(s, a)]$.
6. Temperature update: minimise $\mathbb{E}[-\alpha(\log \pi_\theta(a \mid s) + \mathcal{H}_{\text{target}})]$ in $\log \alpha$.
7. Polyak update: $\phi'_i \leftarrow \tau\phi_i + (1 - \tau)\phi'_i$.

TD3

Hyper-params: as SAC, plus action-noise $\sigma = 0.1$, target-noise $\tilde{\sigma} = 0.2$, noise clip $c = 0.5$, delay $d = 2$.
For each env step:

1. Roll out with $a = \mu_\theta(s) + \mathcal{N}(0, \sigma^2)$.
2. Critic step (twin): target $y = r + \gamma \min_i Q_{\phi'_i}(s', \mu_{\theta'}(s') + \text{clip}(\epsilon, -c, c))$.
3. Every d steps: actor step $(\nabla_\theta \mathbb{E}[Q_{\phi_1}(s, \mu_\theta(s))])$, then polyak update of all targets.

4 Implementation walkthrough

4.1 Tanh-squashed Gaussian (the right way)

```python
class TanhGaussianPolicy(nn.Module):
    def __init__(self, obs_dim, act_dim, hidden=(256, 256)):
        super().__init__()
        # mu and log_sigma heads share a trunk; both depend on s.
        self.trunk = mlp([obs_dim, *hidden], output_activation=nn.ReLU)
        self.mu = nn.Linear(hidden[-1], act_dim)
        self.log_sigma = nn.Linear(hidden[-1], act_dim)

    def sample(self, obs):
        z = self.trunk(obs)
        mu = self.mu(z)
        log_sigma = self.log_sigma(z).clamp(-5, 2)
        sigma = log_sigma.exp()
        eps = torch.randn_like(mu)
        u = mu + sigma * eps
        a = torch.tanh(u)
        # log N(u; mu, sigma^2) - sum log(1 - a^2)  (numerically stable form)
        log_p_u = -0.5 * (((u - mu) / sigma) ** 2 + 2 * log_sigma
                          + math.log(2 * math.pi))
        # log|da/du|^-1 = -log(1 - tanh^2 u); stable form:
        log_det_jac = 2.0 * (math.log(2.0) - u - F.softplus(-2.0 * u))
        log_prob = (log_p_u - log_det_jac).sum(-1)
        return a, log_prob
```

4.2 SAC critic update

```python
def critic_update(batch, actor, q1, q2, q1_tgt, q2_tgt, log_alpha,
                  gamma=0.99, opt=None):
    s, a, r, s2, d = batch
    with torch.no_grad():
        a2, logp2 = actor.sample(s2)
        q_tgt = torch.min(q1_tgt(s2, a2), q2_tgt(s2, a2)) - log_alpha.exp() * logp2
        y = r + gamma * (1 - d) * q_tgt
    L = ((q1(s, a) - y) ** 2 + (q2(s, a) - y) ** 2).mean()
    opt.zero_grad(); L.backward(); opt.step()
```

5 Empirical results

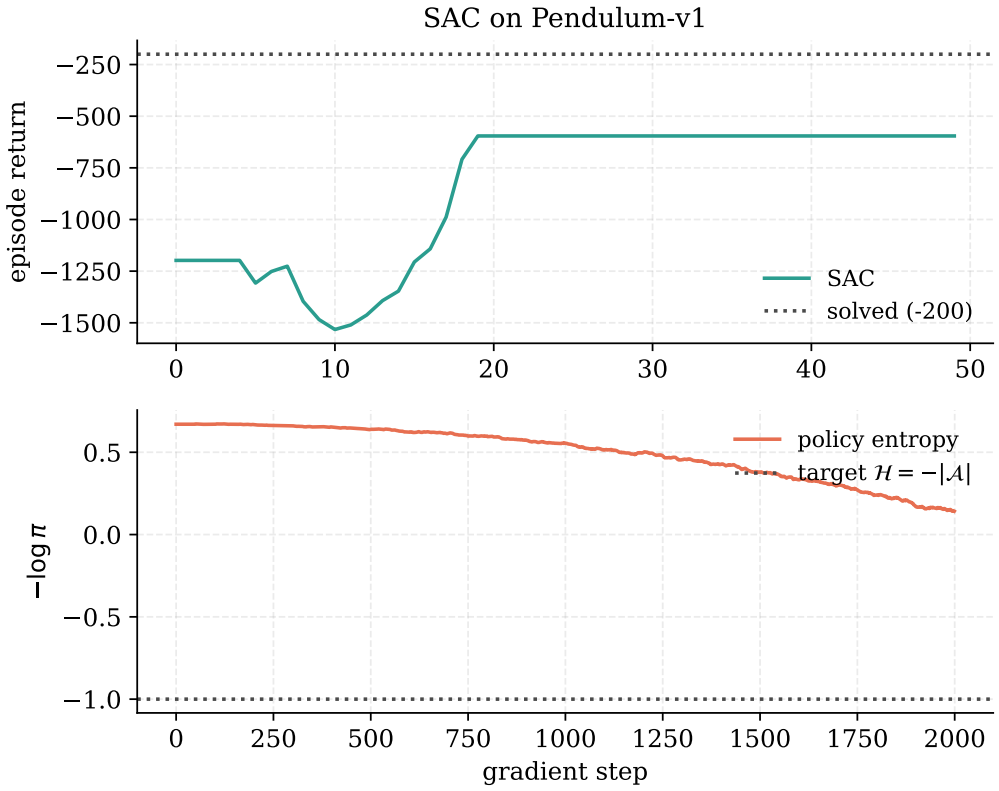


Figure 5.1: SAC on Pendulum-v1. Three seeds; return (top) and policy entropy (bottom). Note that the entropy decreases during training as the policy becomes more confident; with automatic temperature tuning, it stabilises near the target $-|\mathcal{A}| = -1$.

6 Safety angle

Safety angle

Three failure modes that matter for high-stakes off-policy RL.

1. **Q-function over-estimation cascade.** A single mis-fit batch can push $Q(s, a)$ several units too high; the next policy update will increase the probability of that action, generating more high-Q data, generating more upward bias. The twin-critic min trick of TD3/SAC is the structural fix. *Watch the empirical* $\min_i Q_i$ *during training*; a runaway uptrend is the failure signature.

2. **Stuck entropy.** If the automatic temperature update is mis-coded (most commonly: forgetting that the temperature loss should be a function of $\log \alpha$ not α), the policy entropy either explodes or collapses. Mitigation: clamp $\log \alpha \in [-5, 2]$ and assert the gradient sign is correct in unit tests.

3. **Replay-buffer drift.** On long runs ($> 10^6$ steps) the buffer contains transitions from radically different past policies. The Q-targets become biased. This is exactly the setting where *constrained safe RL* (week 16) needs explicit safety filters; the off-policy bias compounds with reward mis-specification to produce non-monotone training.

Matching context. For an off-policy matching system retrained nightly on the day's logs, the entropy-regularised objective is structurally useful: it prevents the policy from collapsing onto a single broker / candidate / item even when the reward signal degenerates. The same maximum-entropy principle is the rationale for the KL-to-reference term in RLHF (week 18); the only difference is that there the "reference" is a fixed prior policy, while in SAC it is the uniform distribution.

7 Matching application

Matching application

SAC as the nightly retrainer for synthetic ATS candidate-ranking.
The matching setting in production usually has continuous-valued match-quality features and is too high-dimensional for tabular methods. SAC fits naturally:

- **State** s**:** candidate embedding (768-dim), job-pool snapshot (256-dim).
- **Action** $a \in \mathbb{R}^d$**:** score adjustments over the top-100 candidates (so $|\mathcal{A}| = 100$, then we softmax-sample the top-K).
- **Reward** r**:** hire-conversion within 30 days, possibly normalised by base rate.
- **Replay buffer** D**:** 90 days of logged (s, a, r, s').

The automatic temperature tuning is particularly valuable here: it removes the need to manually trade off exploration (showing new candidates) vs. exploitation (showing the model's favourites). The maximum-entropy objective keeps the candidate-shortlisting policy diverse, which encourages diversity but provides no individual or group fairness guarantee; an entropy bonus can coexist with severe exposure disparities.

Operational caveats.

- Production deployment requires a CMDP layer (week 16) to enforce hard fairness constraints (e.g. minimum exposure per demographic group). SAC alone is the optimiser; it does not provide guarantees.
- Replay-buffer drift over the 90-day window is a real concern; in practice we re-fit the value head weekly from scratch on the most recent two weeks to combat this.

8 Exercises

E1 ($\star$) Verify the tanh-Jacobian correction: sample $u \sim \mathcal{N}(0,1)$, transform to $a = \tanh u$, compute the corrected $\log \pi(a)$, and check that the density $\exp(\log \pi(a))$ integrates to 1 over $(-1, 1)$.

E2 ($\star$) Derive the soft-optimal policy in closed form via the Lagrangian and confirm $\pi^\star \propto \exp(Q/\alpha)$.

E3 ($\star\star$) Implement the SAC critic update for Pendulum and verify the loss decreases over 200 steps.

E4 ($\star\star$) Implement the actor reparameterised loss; verify the gradient sign on a toy 1-d problem.

E5 ($\star\star$) Implement automatic temperature tuning; show that α converges to a value that yields $\mathcal{H} \approx \mathcal{H}_{\text{target}}$.

E6 ($\star\star$) Train SAC on Pendulum-v1 to a smoothed return ≥ -200 within 8000 env steps.

E7 ($\star\star\star$) Code TD3 in $\sim$100 lines; compare wall-clock to SAC on Pendulum.

E8 ($\star\star\star$, meaningful objective) For a logged-bandit matching dataset, run an off-policy SAC variant: load the logs into the replay buffer and train without env interaction. Verify that the resulting policy beats the behaviour policy in SNIPS-estimated value.

9 Further reading

See `extended_reading.pdf`. Five papers: Lillicrap et al. 2015 (DDPG), Fujimoto et al. 2018 (TD3), Haarnoja et al. 2018a/b (SAC + automatic temperature), Bhatt et al. 2024 (CrossQ), Lee et al. 2025 (SimBa). Sourced from `research/deeprl_core_sota.md` §3.1–3.7.